COMPUTER VISION

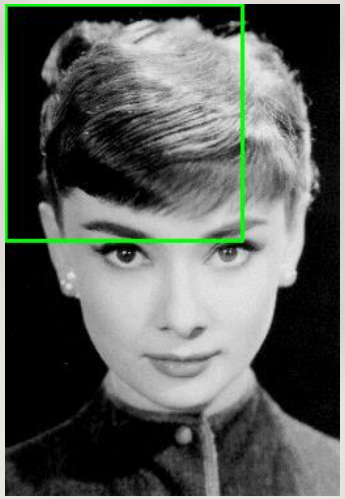
CLASSIFICATION

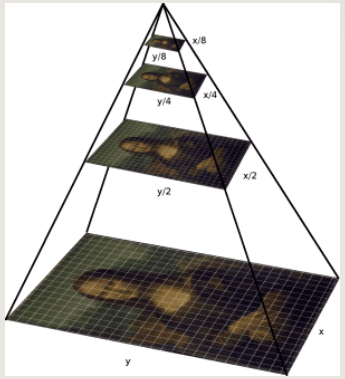
Ngo Quoc Viet-2024

Contents

Lecture learning outcomes

- Understanding Haar and HOG features in face recognition and pedestrian detection
- Using pretrained cascade classifier to implement face recognition in image or video
- Implementing HOG features
- Implementing pedestrian detection with HOG features and Linear SVM classifier.





Face Detection using Haar Cascades

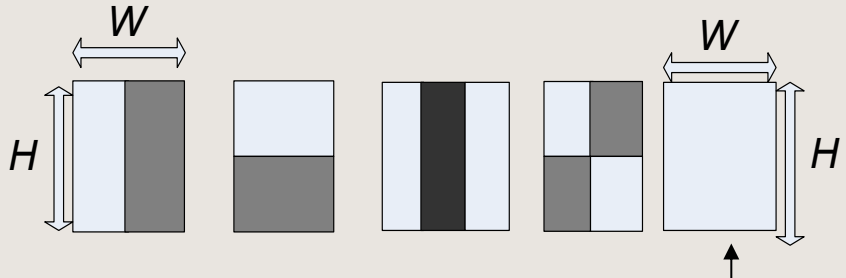
Source: https://www.wired.com/story/behind-rise-chinas-facial-recognition-giants/

Face Detection using Haar Cascades

- Viola, Paul, and Michael J. Jones. *Rapid Object Detection using a Boosted Cascade of Simple Features*. 2001.
- Viola, Paul, and Michael J. Jones. *Robust real-time face detection*. International journal of computer vision 57.2 (2004): 137-154.
- Two above works use Haar feature-based cascade classifiers.
- Haar-like features are simple digital image features that were introduced in a real-time face detector.
- These features can be efficiently computed on any scale in constant time, using an integral image.

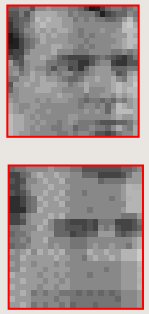
Haar-like features and rectangle sum

- The Haar-like feature descriptors come into different types (edge features, line features and four-rectangle features)

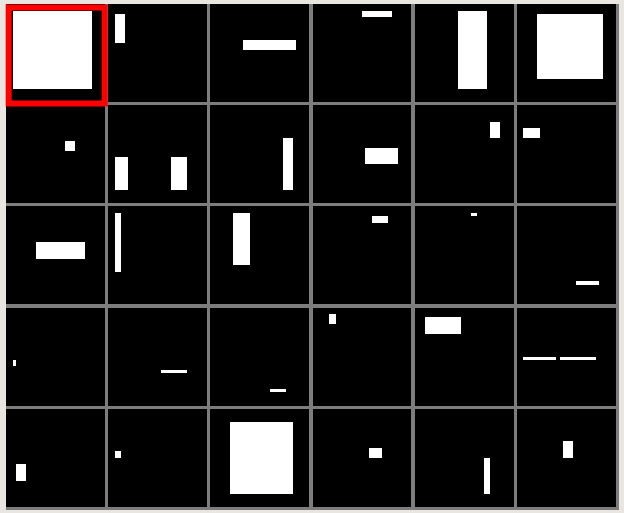


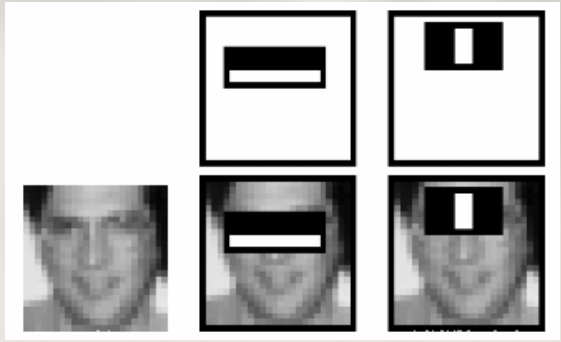
$W=H=4$

Sum of pixels in a rectangle: rectangle sum



$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & -1 & -1 & -1 \\ 1 & 1 & -1 & -1 & 0 & 0 \\ 0 & 0 & 1 & 1 & -1 & -1 \end{bmatrix} \vec{X}_6$$

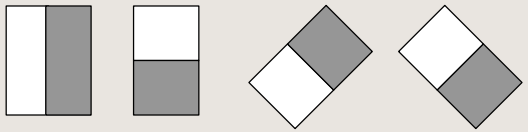




Haar features are just like our convolutional kernel. Each feature is a single value obtained by subtracting sum of pixels under the white rectangle from sum of pixels under the black rectangle

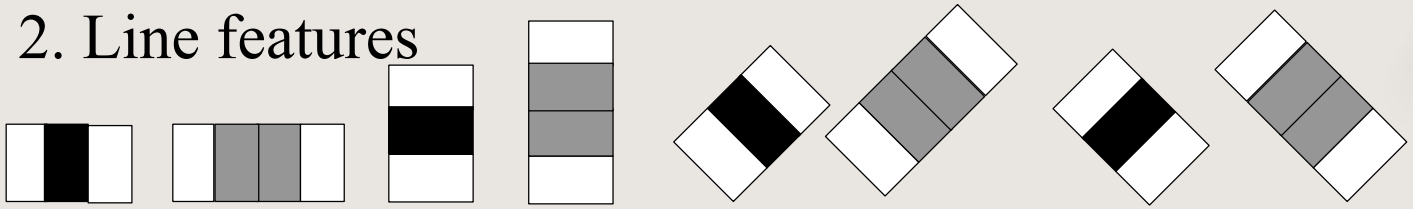
Extended Haar-like features

1. Edge features



(a) (b) (c) (d)

2. Line features



(a) (b) (c) (d) (e) (f) (g) (h)

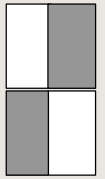
3. Center-surround features

(a) (b)

4. Box features

(a) (b)

5. Feature in [1]



Rectangle sum - definition

- A rectangle in the image $X(j_1, j_2)$ is specified by:
- $rect = (j_1, j_2, N_1, N_2, \theta)$,
  - (j_1, j_2): upper left position
  - (N_1, N_2): size of the rectangle.
  - θ=0°: upright rectangle
  - θ=45°: 45° rotated rectangle.

$$RectSum(rect) = \sum_{v=j_1}^{j_1+N_1-1} \sum_{u=j_2}^{j_2+N_2-1} X(u,v)$$

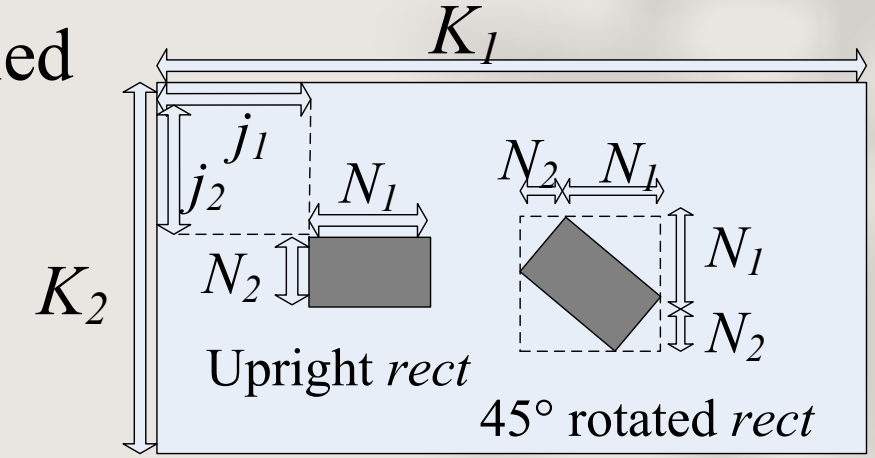
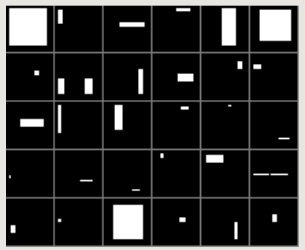


Image $X(j_1, j_2)$



- Related to *dc* component

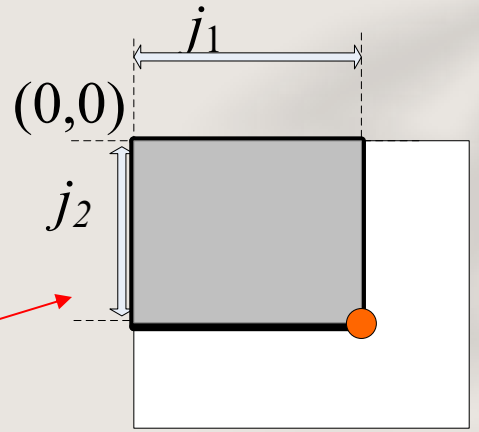
$$\frac{1}{Scale} RectSum(rect) = \frac{1}{Scale} \sum_{v=j_1}^{j_1+N_1-1} \sum_{u=j_2}^{j_2+N_2-1} X(u,v)$$

N_1N_2-1 additions → 3 additions

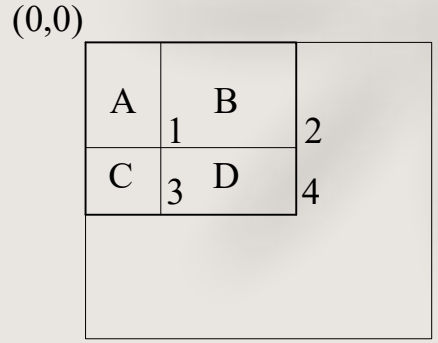
Integral image

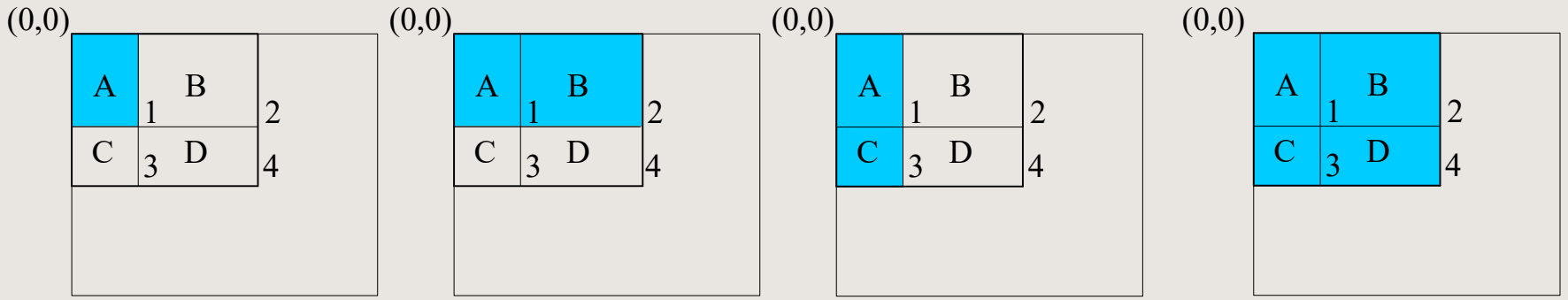
- The integral image $I(j_1, j_2)$ is the sum of pixels above and to the left of (j_1, j_2)

$$I(j_1, j_2) = \sum_{u=0}^{j_1-1} \sum_{v=0}^{j_2-1} X(u,v)$$



- 1: A 2: A+B
- 3: A+C 4: A+B+C+D
- A-D: rectangle sum

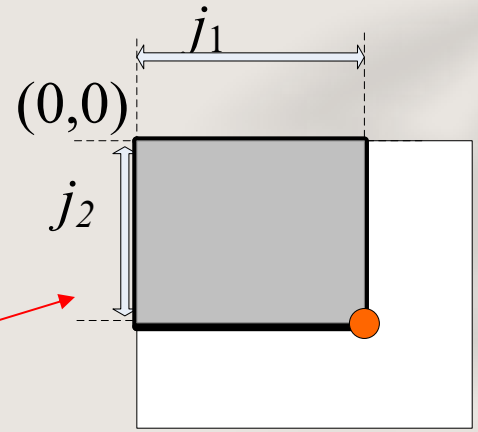




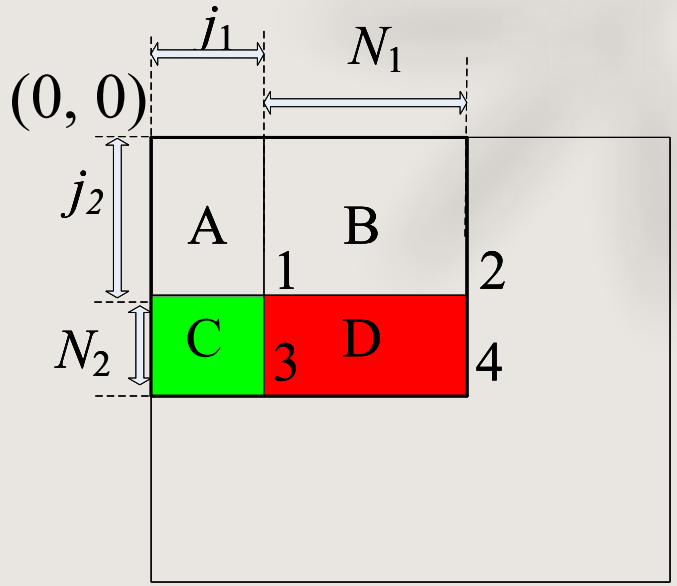
Integral image method

- The integral image $I(j_1, j_2)$ is the sum of pixels above and to the left of (j_1, j_2)

$$I(j_1, j_2) = \sum_{u=0}^{j_1-1} \sum_{v=0}^{j_2-1} X(u,v)$$



- 1: A 2: A+B
- 3: A+C 4: A+B+C+D
  - D = 4 + 1 − 3 − 2
    = (4 − 2) − (3 − 1)
  - A-D: rectangle sum



3 additions

$$RectSum(rect) = \sum_{u=j_1}^{j_1+N_1-1} \sum_{v=j_2}^{j_2+N_2-1} X(u,v)$$

$$= I(j_1 + N_1, j_2 + N_2) + I(j_1, j_2) - I(j_1, j_2 + N_2) - I(j_1 + N_1, j_2).$$

4 1 3 2

Haar-like feature descriptor

```python
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import haar_like_feature_coord
from skimage.feature import draw_haar_like_feature
images = [np.zeros((2, 2)), np.zeros((2, 2)),
          np.zeros((3, 3)), np.zeros((3, 3)),
          np.zeros((2, 2))]
feature_types = ['type-2-x', 'type-2-y',
                 'type-3-x', 'type-3-y',
                 'type-4']
fig, axs = plt.subplots(3, 2)
for ax, img, feat_t in zip(np.ravel(axs), images, feature_types):
    coord, _ = haar_like_feature_coord(img.shape[0], img.shape[1], feat_t)
    haar_feature = draw_haar_like_feature(img, 0, 0,
                                          img.shape[0],
                                          img.shape[1],
                                          coord,
                                          max_n_features=1,
                                          random_state=0)
    ax.imshow(haar_feature)
    ax.set_title(feat_t)
    ax.set_xticks([])
    ax.set_yticks([])
fig.suptitle('The different Haar-like feature descriptors')
```

https://scikit-image.org/docs/stable/auto_examples/features_detection/plot_haar.html

https://scikit-image.org/docs/stable/auto_examples/applications/plot_haar_extraction_selection_classification.html

Pre-trained Haar Cascades

- The OpenCV library manages a repository on GitHub for all popular haar cascades pre-trained files that can be used for various object detection tasks, for example
  - Human face detection
  - Eye detection
  - Vehicle detection
  - Nose / Mouth detection
  - Body detection
  - License plate detection
- https://github.com/opencv/opencv/tree/master/data/haarcascades

Face detection with Pre-trained Haar Cascades

```
while cap.isOpened():
    _, frame = cap.read()
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = faceDetector.detectMultiScale(gray, 1.1, 4)
    for (x,y, w, h) in faces:
        cv2.rectangle(frame, pt1 = (x,y),pt2 = (x+w, y+h), color = (255,0,0),thickness =  3
        roi_gray = gray[y:y+h,x:x+w]
        roi_color = frame[y:y+h, x:x+w]
        eyes = eyeDectector.detectMultiScale(roi_gray, 1.5, 9)
        for (ex, ey, ew, eh) in eyes:
            cv2.rectangle(roi_color, (ex, ey), (ex + ew, ey + eh), (0, 255, 0), 5)
    cv2.imshow("window", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
frame.release()
```

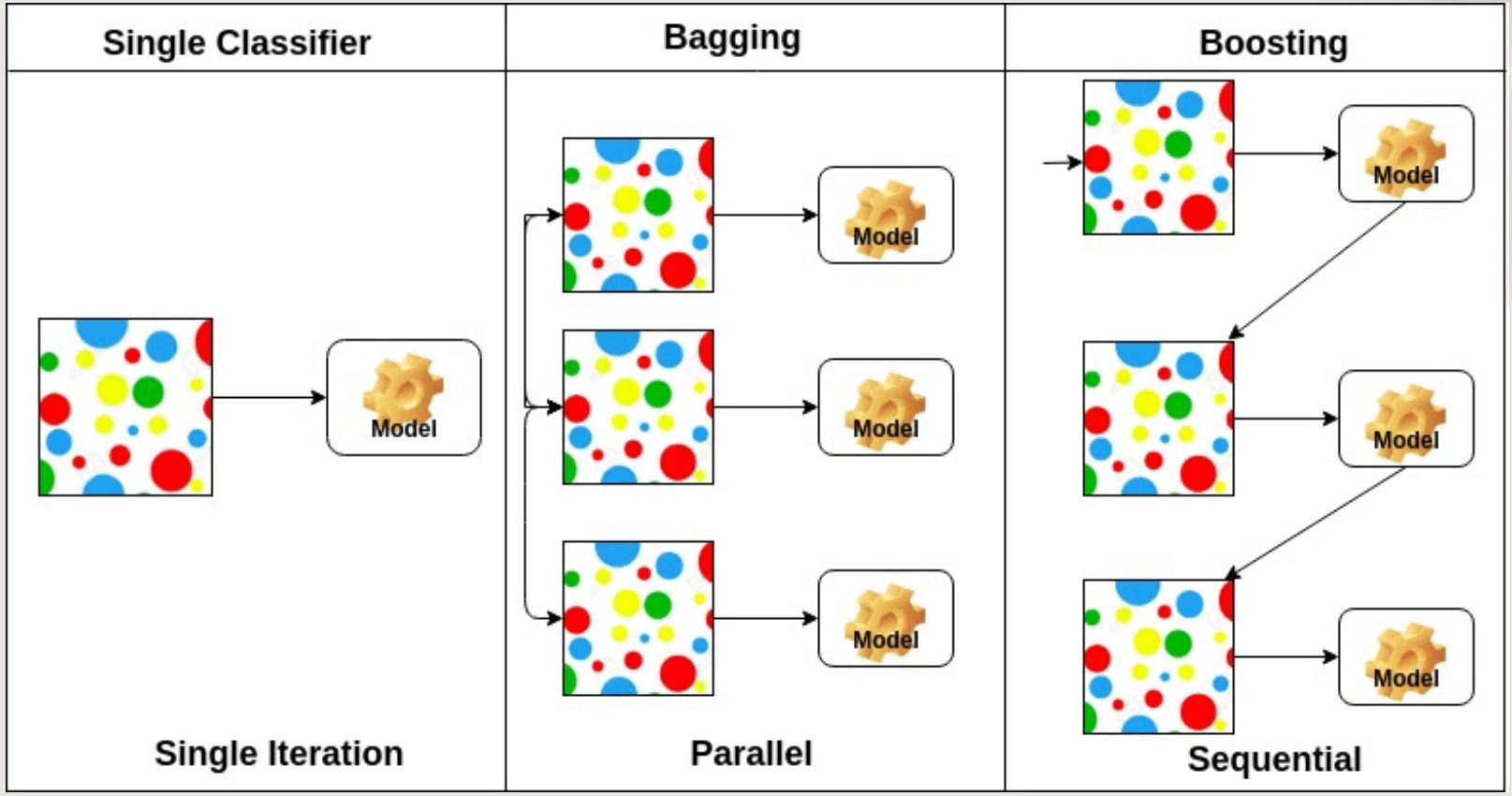
Brief of Cascade Classifier

- Cascading is a particular case of ensemble learning based on the concatenation of several classifier, using all information collected from the output from a given classifier as additional information for the next classifier in the cascade.
- Cascading classifiers are trained with positive sample views of a particular object and arbitrary negative images of the same size.
- Cascades are usually done through cost-aware AdaBoost.

Boosting algorithms

- 1990 – Boost-by-majority algorithm (Freund)
- 1995 – AdaBoost (Freund & Schapire)
- 1997 – Generalized version of AdaBoost (Schapire & Singer)
- 2001 – AdaBoost in Face Detection (Viola & Jones)

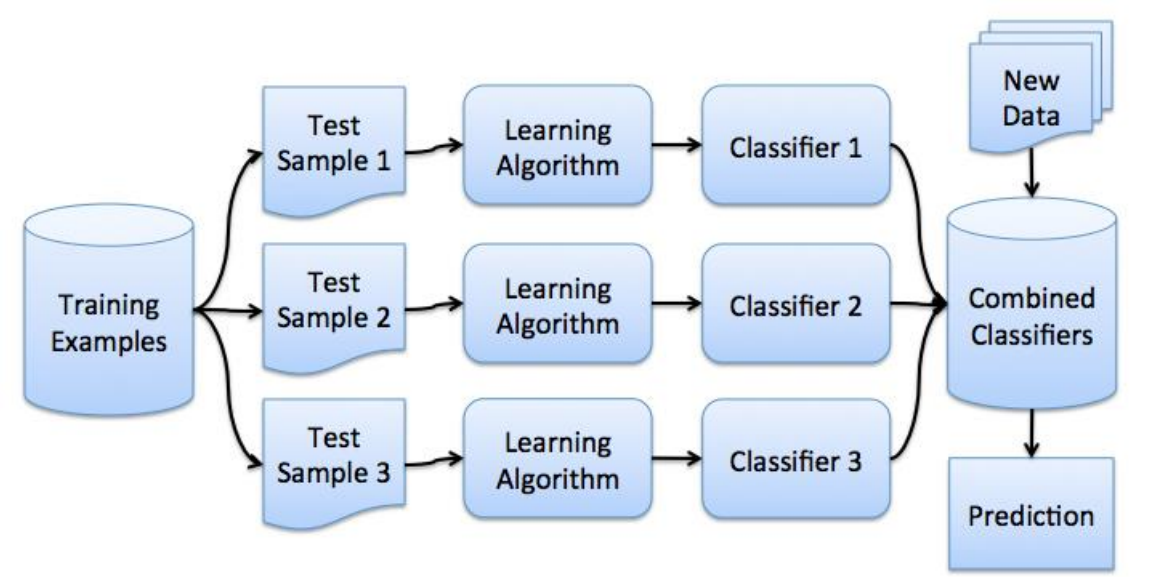
Boosting & Bagging



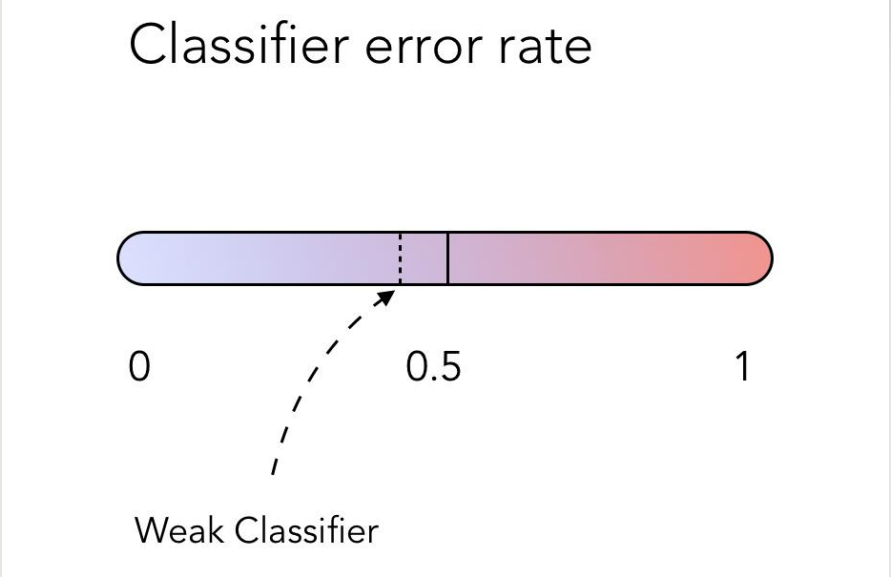
https://www.datacamp.com/community/tutorials/adaboost-classifier-python

Boosting ALGORITHMS

- Boosting trains a series a low performing algorithms, called weak learners, by adjusting the error metric over time.
- Weak learners are algorithms whose error rate is slightly under 50% as illustrated below.



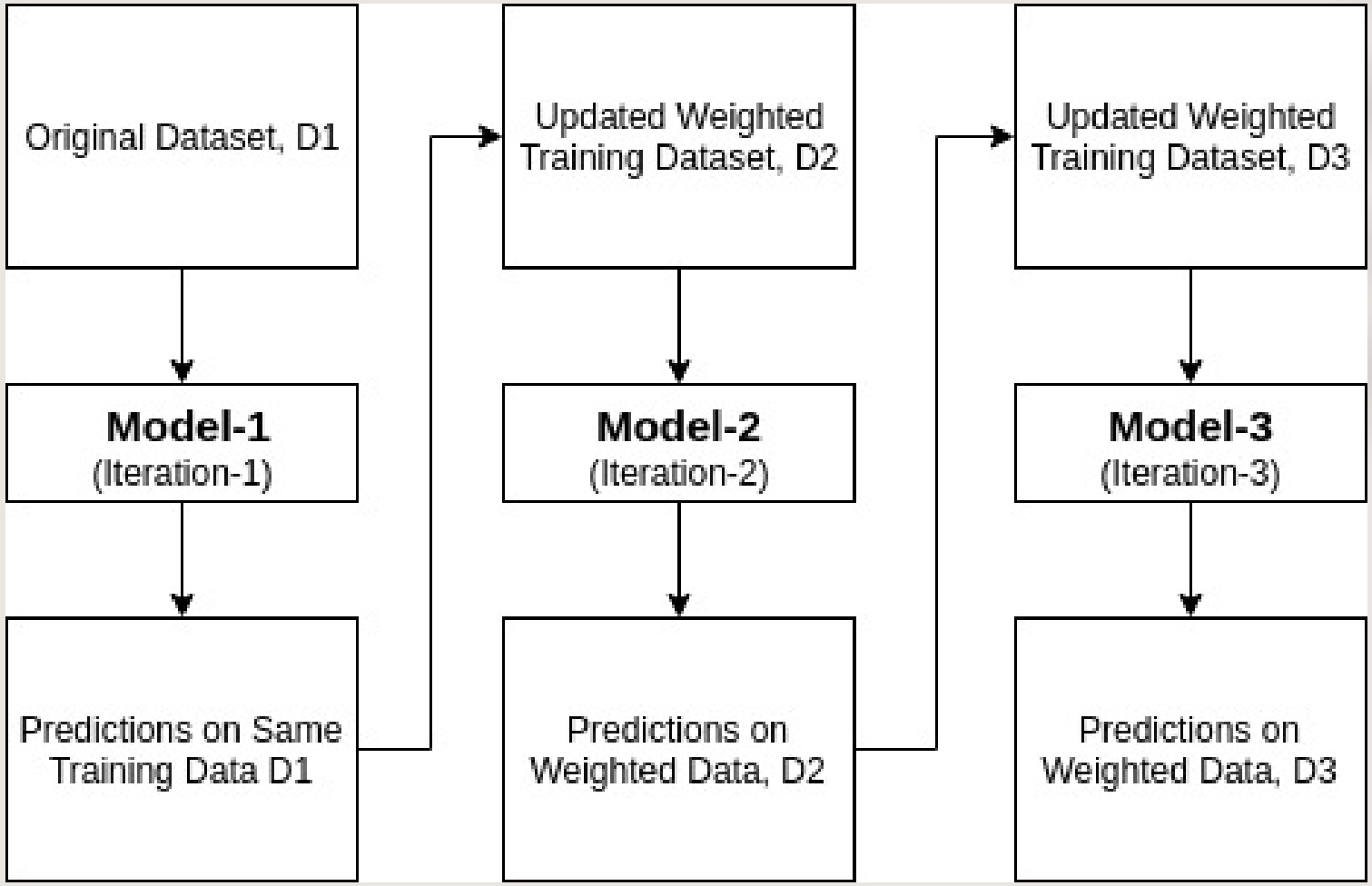
https://prachimjoshi.wordpress.com/2015/07/23/bagging-and-boosting/



AdaBoost

- AdaBoost = Adaptive Boosting (Schapire 1995)
- AdaBoost is an algorithm for constructing a ″strong″ classifier as linear combination of ″simple″ ″weak″ classifiers.

$$H(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$$


Original Dataset, D1
Updated Weighted Training Dataset, D2
Updated Weighted Training Dataset, D3
Model-1 (Iteration-1)
Model-2 (Iteration-2)
Model-3 (Iteration-3)
Predictions on Same Training Data D1
Predictions on Weighted Data, D2
Predictions on Weighted Data, D3

Algorithm 2 : AdaBoost.M1

1 **Input :** Dataset $D = \{(a_1, c_1), (a_2, c_2), \ldots, (a_N, c_N)\}$,
Base learner L and
Number of learning iteration T

2 Initialize equal weight to all training samples $w_i = 1/N$, $i = 1, 2, \ldots, N$

3 For $t = 1$ to T :

(a) Train a base learner h_t from D using D_t to training sample using w_i.
$h_t = L(D, D_t)$

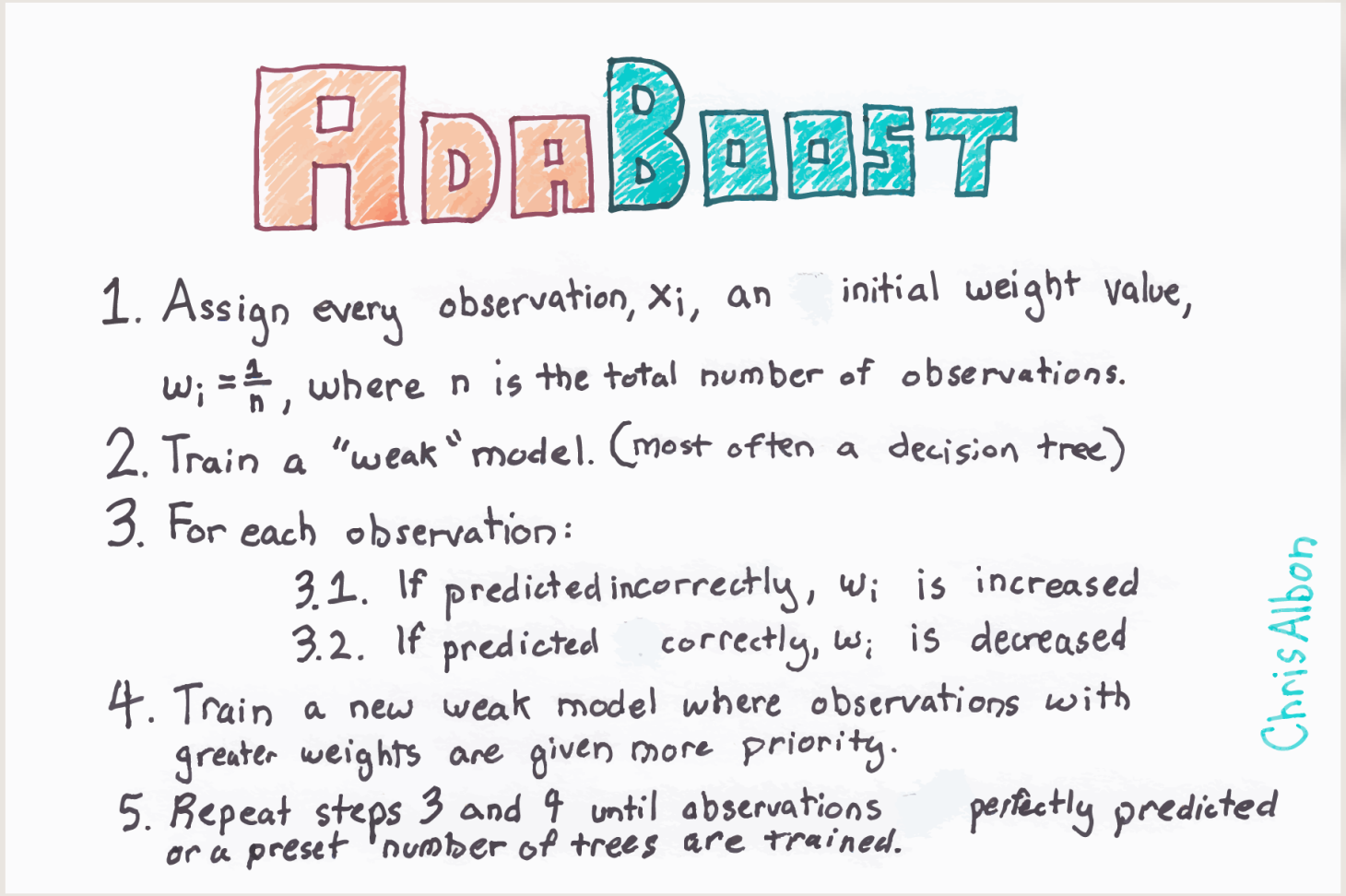
(b) Compute error of h_t as
$$err_t = \frac{\sum_{i=1}^{N} w_i I(h_t(a_i) \neq c_i)}{\sum_{i=1}^{N} w_i}$$

(c) Compute the weight of h_t as
$$\alpha_t = \log\left(\frac{1 - err_t}{err_t}\right)$$

(d) Set $w_i \leftarrow w_i . \exp[\alpha_t I(h_t(a_i) \neq c_i)]$

4 **Output :** $H(a) = \text{sign} \sum_{t=1}^{T} \alpha_t h_t(a)$

https://link.springer.com/chapter/10.1007/978-981-15-0633-8_22



https://chrisalbon.com/machine_learning/trees_and_forests/adaboost_classifier/

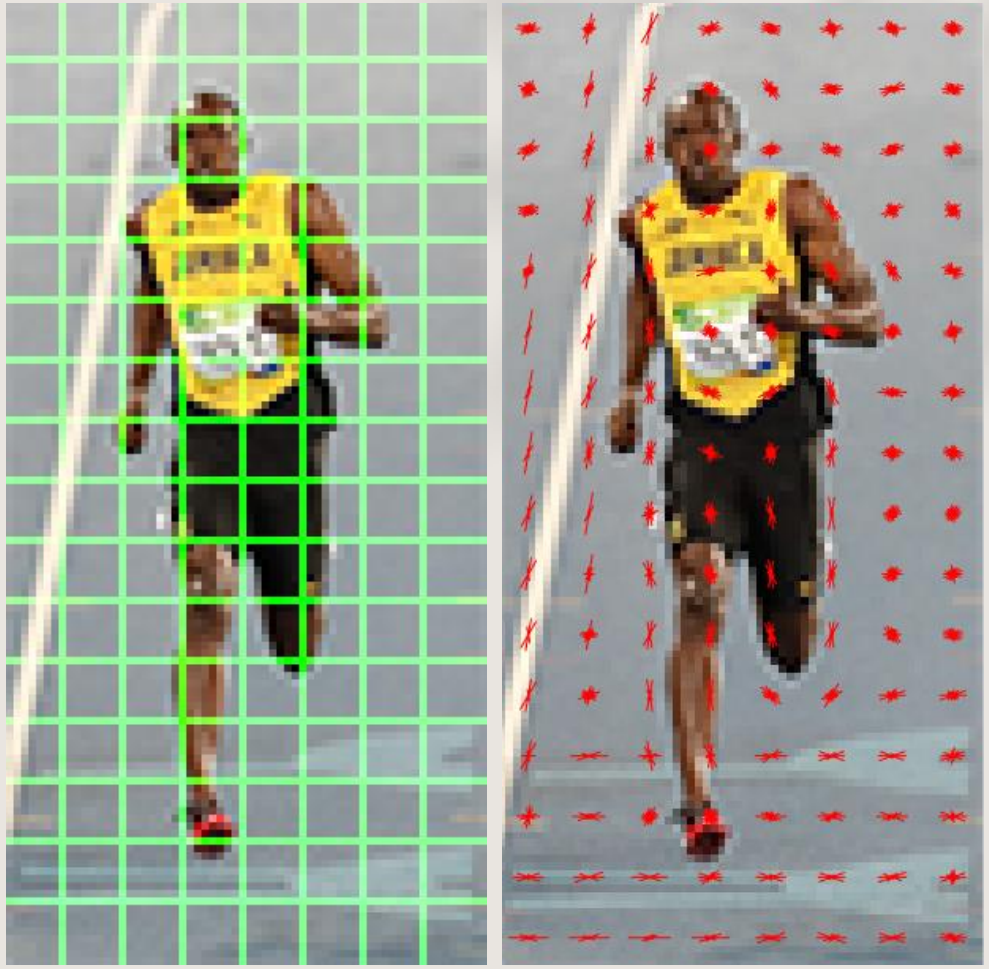
Adaboost explained& simple code

1. https://www.youtube.com/watch?v=LsK-xG1cLYA.
2. https://www.mygreatlearning.com/blog/adaboost-algorithm/
3. https://www.analyticsvidhya.com/blog/2021/09/adaboost-algorithm-a-complete-guide-for-beginners/

4. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.AdaBoostClassifier.html
5. https://www.datacamp.com/community/tutorials/adaboost-classifier-python

Histogram of Oriented Gradients

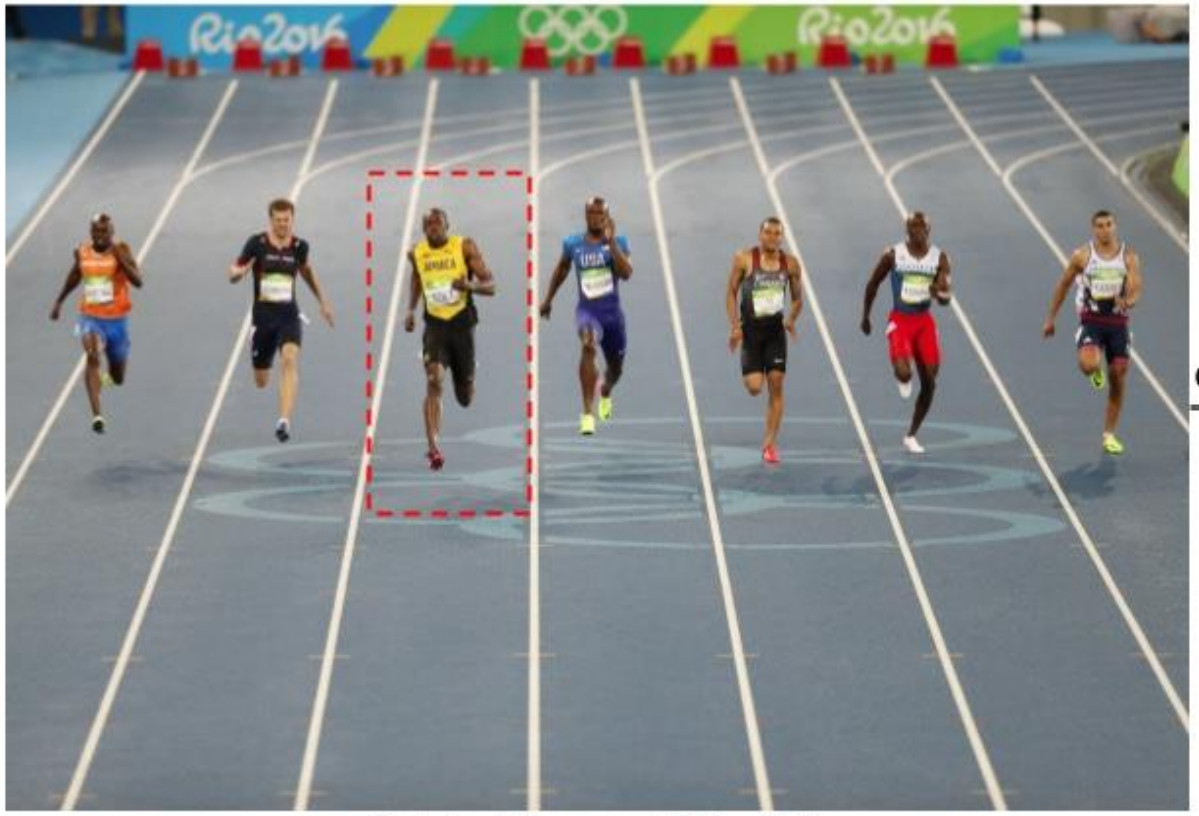
- A HOG (Histogram of Oriented Gradients) feature is a feature descriptor used in computer vision and image processing for object detection and recognition.
- It analyzes the distribution of edge orientations within an object to describe its shape and appearance.
- The HOG method involves computing the gradient magnitude and orientation for each pixel in an image and then dividing the image into small cells.

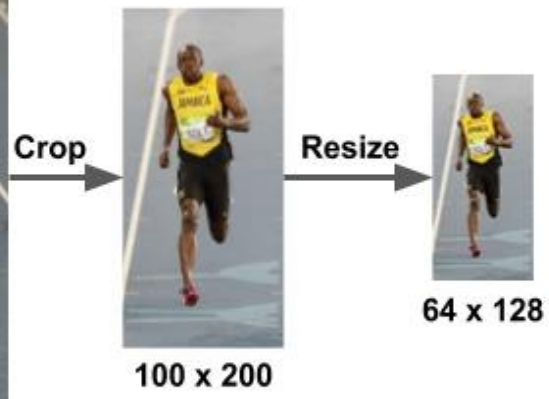


Steps to calculate HOG

- Step 1: Preprocessing.
- Step 2: Calculate the Gradient Images.
- Step 3: Calculate Histogram of Gradients in 8×8 cells
- Step 4: 16×16 Block Normalization
- Step 5: Calculate the Histogram of Oriented Gradients feature vector.

HOG - Preprocessing





https://learnopencv.com/histogram-of-oriented-gradients/

- For pedestrian detection, we need to preprocess the image and bring down the width to height ratio to 1:2. For example, image size in pedestrian detection is 64 x 128.
- This is because we will be dividing the image into 8*8 and 16*16 patches to extract the features.

HOG - Calculate the Gradient Images

- Calculate the **gradient for every pixel** in the image. Gradients are the **small change in the x and y directions**. For example, we generate the below pixel matrix for the given patch

121	10	78	96	125
48	152	68	125	111
145	78	85	89	65
154	214	56	200	66
214	87	45	102	45

- Change in X direction (g_x) = 89 – 78 = 11
- Change in Y direction (g_y) = 68 – 56 = 8

- The same process is repeated for all the pixels in the image
- This process will give us two new matrices – one storing gradients in the x-direction and the other storing gradients in the y direction.

- This is easily achieved by filtering the image with the **Sobel** operator in OpenCV with kernel size 1.

```
gx = cv.Sobel(img, cv.CV_64F, 1, 0, ksize=1)
gy = cv.Sobel(img, cv.CV_64F, 0, 1, ksize=1)
```

HOG - Calculate the Gradient Images

- Using the gradients in the previous step, we will now determine the *magnitude* and *direction* for each pixel value.

$$g = \sqrt{g_x^2 + g_y^2}, \theta = arctan\frac{g_y}{g_x}$$

121	10	78	96	125
48	152	68	125	111
145	78	85	89	65
154	214	56	200	66
214	87	45	102	45

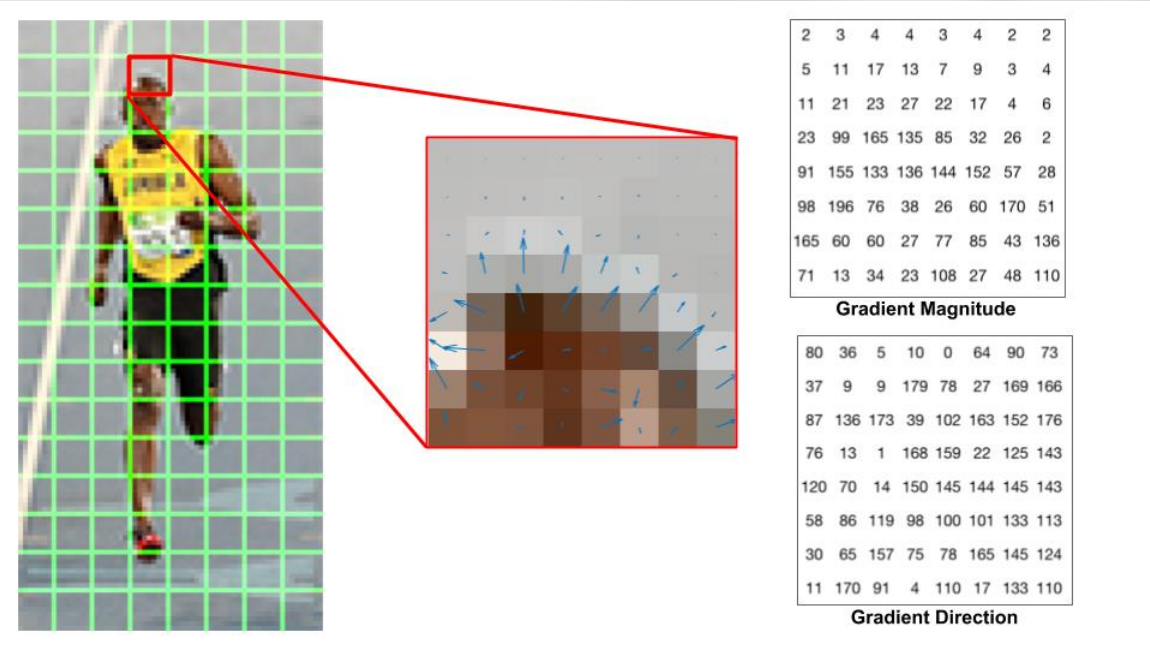
- $g = \sqrt{11^2 + 8^2} = 13.6$
- $\theta = atan\left(\frac{8}{11}\right) = 0.628796286\, rad \sim 36°$

- It is easy to calculate magnitude and direction.

```
g = np.sqrt(gx**2+gy**2)                    #magnitude
theta = np.arctan2(gy, gx+0.0001)** (180/ np.pi)    #direction
```

HOG-Histogram of Gradients in 8×8 cells

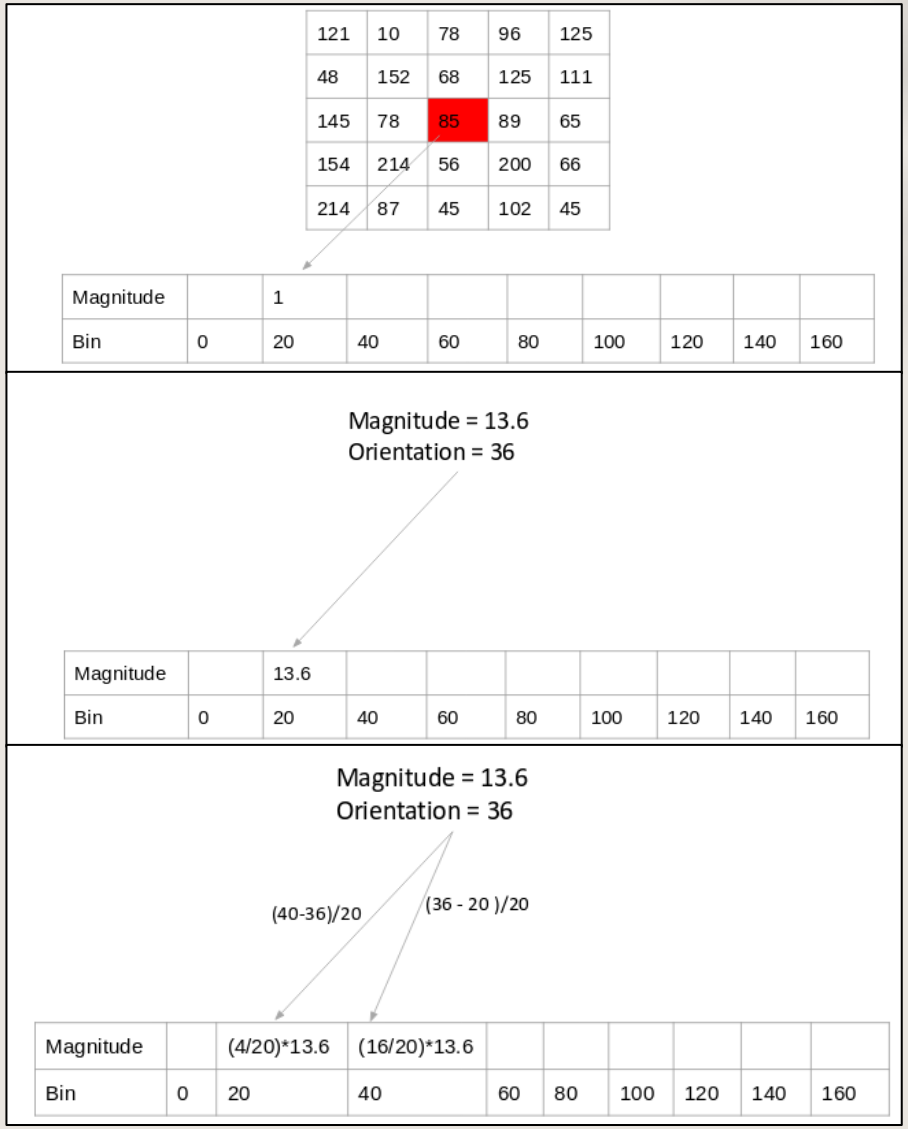
- The image is divided into 8×8 cells and a histogram of gradients is calculated for each 8×8 cells.
- The gradient of this patch contains 2 values (magnitude and direction) per pixel which adds up to 8x8x2 = 128 numbers.
- A bin is selected based on the direction, and the vote (the value that goes into the bin) is selected based on the magnitude.
- The histogram is an array of 9 **bins** corresponding to angles 0, 20, 40, 60 … 160.



Why 8×8 patch ? Why not 32×32 ? It is a design choice informed by the scale of features we are looking for. HOG was used for pedestrian detection initially. 8×8 cells in a photo of a pedestrian scaled to 64×128 are big enough to capture interesting features (e.g. the face, the top of the head etc.).

HOG-Histogram of Gradients in 8×8 cells

- For each pixel, we will check the orientation, and choose 1 of 3 methods and store value to a bin.
  - The frequency of the orientation values
  - Adding the gradient magnitude
  - The contribution of a pixel's gradient to the bins on either side of the pixel gradient
- If we divide the image into 8×8 cells and generate the histograms, we will get a 9 x 1 matrix for each cell.



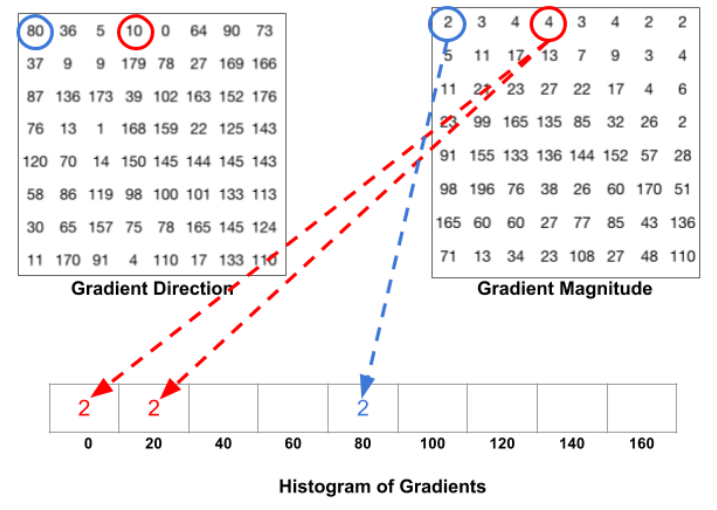
Frequency of the orientation values

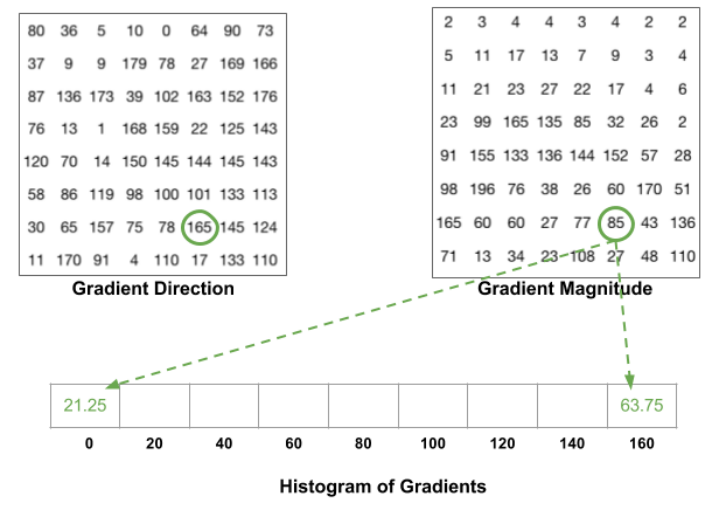
The gradient magnitude

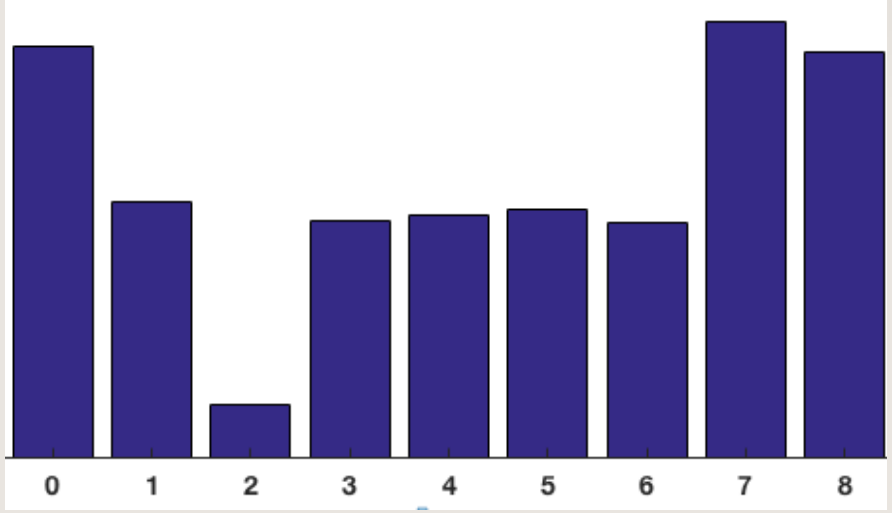
The contribution of a pixel's gradient to the bins on either side of the pixel gradient

HOG-Histogram of Gradients in 8×8 cells

- It has an angle of 80 degrees and magnitude of 2. So it adds 2 to the 5th bin. The gradient at the pixel encircled using red has an angle of 10 degrees and magnitude of 4. Since 10 degrees is half way between 0 and 20, the vote by the pixel splits evenly into the two bins.
- If the angle is greater than 160 degrees, it is between 160 and 180, and we know the angle wraps around making 0 and 180 equivalent. So in the example below, the pixel with angle 165 degrees contributes proportionally to the 0 degree bin and the 160 degree bin.
- The contributions of all the pixels in the 8×8 cells are added up to create the 9-bin histogram







HOG-Histogram of Gradients in 8×8 cells

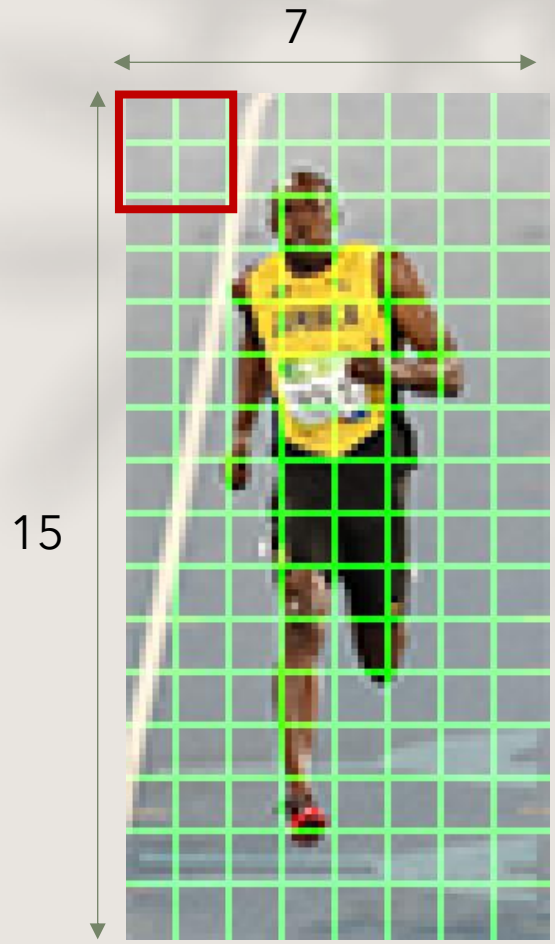
```
# histogram
magnitude = np.sqrt(np.square(dx) + np.square(dy))
orientation = np.arctan(np.divide(dy, dx + 0.00001))  # radian
orientation = np.degrees(orientation)  # -90 -> 90
orientation += 90  # 0 -> 180

num_cell_x = w // cell_size  # 8
num_cell_y = h // cell_size  # 16
hist_tensor = np.zeros([num_cell_y, num_cell_x, bins])  # 16 x 8 x 9
for cx in range(num_cell_x):
    for cy in range(num_cell_y):
        ori = orientation[cy * cell_size:cy * cell_size + \
                          cell_size, cx * cell_size:cx * cell_size + cell_size]
        mag = magnitude[cy * cell_size:cy * cell_size + cell_size, \
                          cx * cell_size:cx * cell_size + cell_size]
        hist, _ = np.histogram(ori, bins=bins, range=(0, 180), weights=mag)  # 1-
        hist_tensor[cy, cx, :] = hist
```

HOG-16×16 Block Normalization

- The gradients of the image are sensitive to the overall lighting. If you make the image darker by dividing all pixel values by 2, the gradient magnitude will change by half, and therefore the histogram values will change by half.
- We can reduce this lighting variation by normalizing the gradients.
- By combining four 8×8 cells to create a 16×16 block → single 36×1 matrix.
- To normalize this matrix, we will divide each of these values by the square root of the sum of squares of the values.

HOG-16×16 Block Normalization

```
# normalization
redundant_cell = block_size - 1
feature_tensor = np.zeros(
    [num_cell_y - redundant_cell, num_cell_x - redundant_cell, \
     block_size * block_size * bins])
for bx in range(num_cell_x - redundant_cell):  # 7
    for by in range(num_cell_y - redundant_cell):  # 15
        by_from = by
        by_to = by + block_size
        bx_from = bx
        bx_to = bx + block_size
        v = hist_tensor[by_from:by_to, bx_from:bx_to, :].flatten()  # to 1-D array
        feature_tensor[by, bx, :] = v / LA.norm(v, 2)
        # avoid NaN:
        if np.isnan(feature_tensor[by, bx, :]).any():  # avoid NaN (zero division)
            feature_tensor[by, bx, :] = v
```



There are 15x7x36=3780 numbers for 64x128 image

One line to implement HOG in Python.

fd, hog_image = hog(resized_img, orientations=9, pixels_per_cell=(8, 8), cells_per_block=(2, 2), visualize=True, multichannel=True)

Pedestrian detection with HOG - Step by Step

- Scan image(s) at all scales and locations
- Extract features over windows
- Run linear SVM classifier on all locations
- Fuse multiple detections in 3D position & scale space
- Object detections with bounding boxes

Scale-down the image and slide the window again (the size of the window is always the same

```
def sliding_window(image, window_size, step_size):
    for y in range(0, image.shape[0], step_size[1]):
        for x in range(0, image.shape[1], step_size[0]):
            yield (x, y, image[y: y + window_size[1], x: x + window_size[0]])
```

Dataset: https://huggingface.co/datasets/marcelarosalesj/inria-person/tree/main/data_ped.

Pedestrian detection with HOG - Step by Step

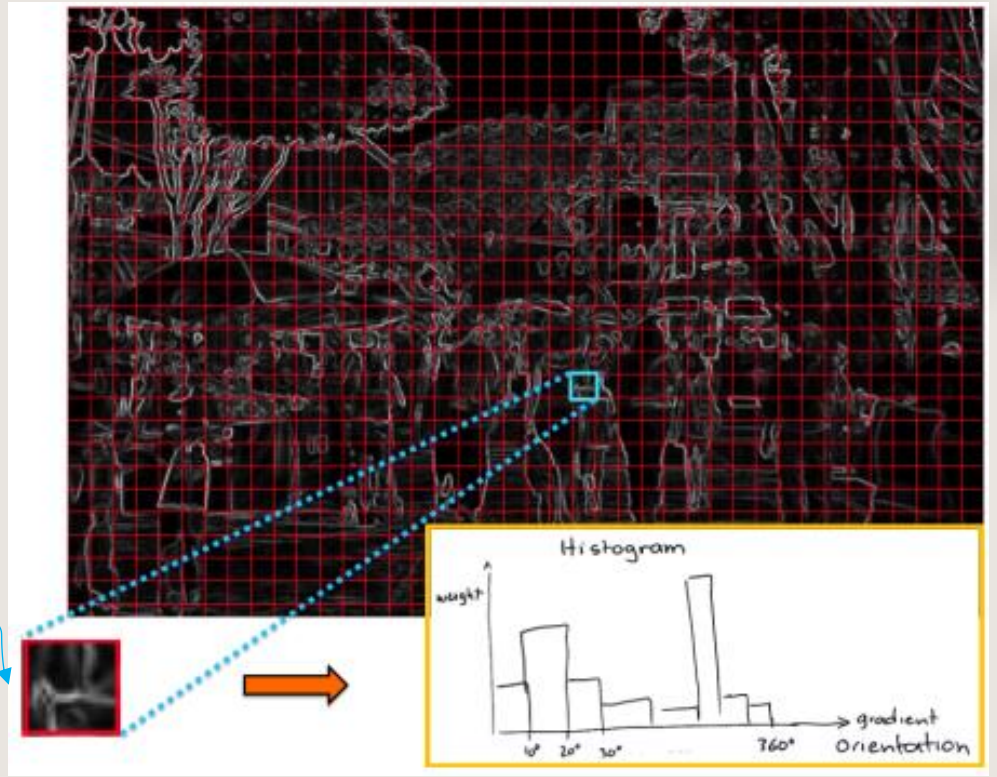
- Scan image(s) at all scales and locations
- Extract features over windows
- Run linear SVM classifier on all locations
- Fuse multiple detections in 3D position & scale space
- Object detections with bounding boxes

Compute gradients → Weighted vote into orientation bin cells → Normalize over overlapping blocks

Divide the image into cells of 8 × 8 pixels.

Compute HOG for each 8x8 cell

9-dim feature vector for each cell

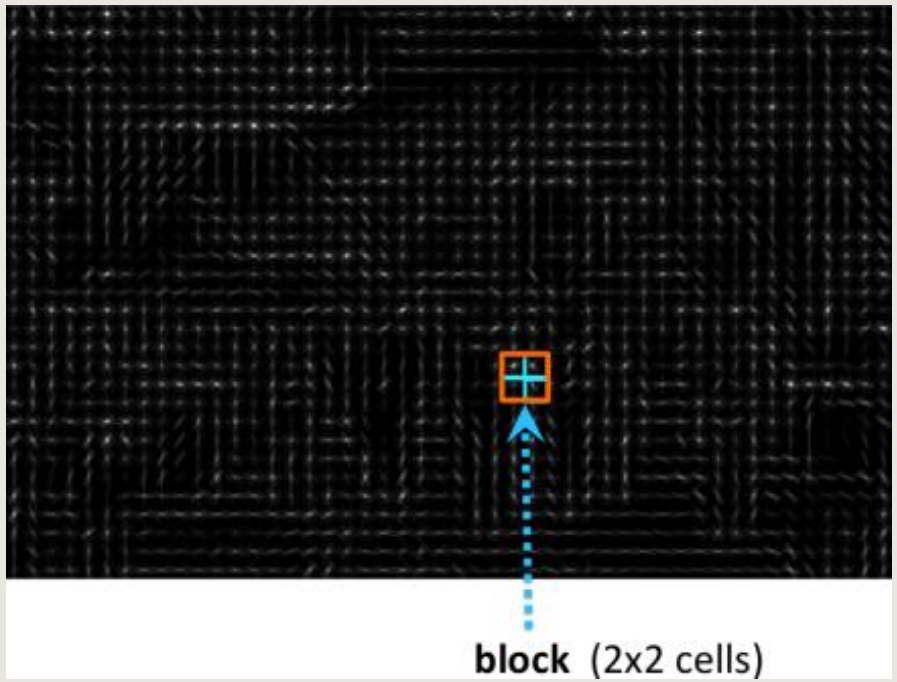


Pic from: Sanja Fidler

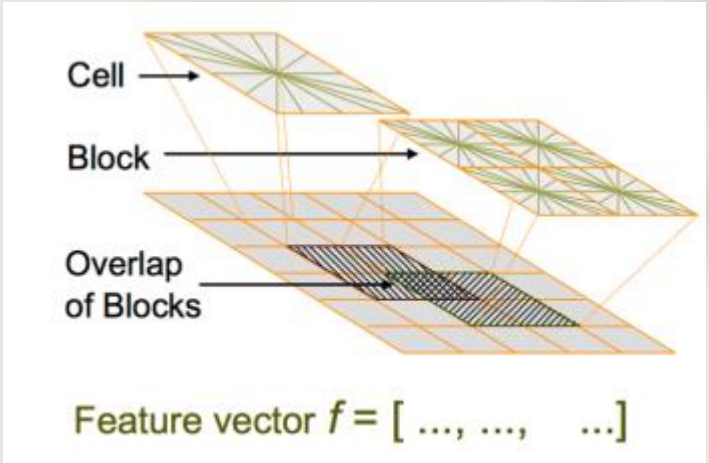
Pedestrian detection with HOG - Step by Step

- Scan image(s) at all scales and locations
- Extract features over windows
- Run linear SVM classifier on all locations
- Fuse multiple detections in 3D position & scale space
- Object detections with bounding boxes

Compute gradients → Weighted vote into orientation bin cells → Normalize over overlapping blocks



Pic from: Sanja Fidler



L2 normalization for each block

fd, hog_image = hog(resized_img, orientations=9, pixels_per_cell=(8, 8), cells_per_block=(2, 2), visualize=True, multichannel=True)

Pedestrian detection with HOG - Step by Step

Scan image(s) at all scales and locations

Extract features over windows

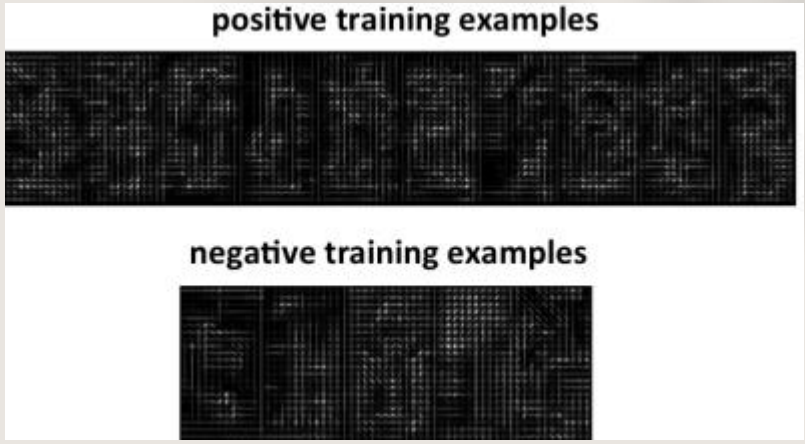
Run linear SVM classifier on all locations

Fuse multiple detections in 3D position & scale space

Object detections with bounding boxes

Train classifier

Predict yes/no of object class in each image window



Pic from: Sanja Fidler

Train classifier. In this lecture, we use Linear SVM

```
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2)
model = LinearSVC()
model.fit(X_train, y_train)
```

Pedestrian detection with HOG - Step by Step

Scan image(s) at all scales and locations

Extract features over windows

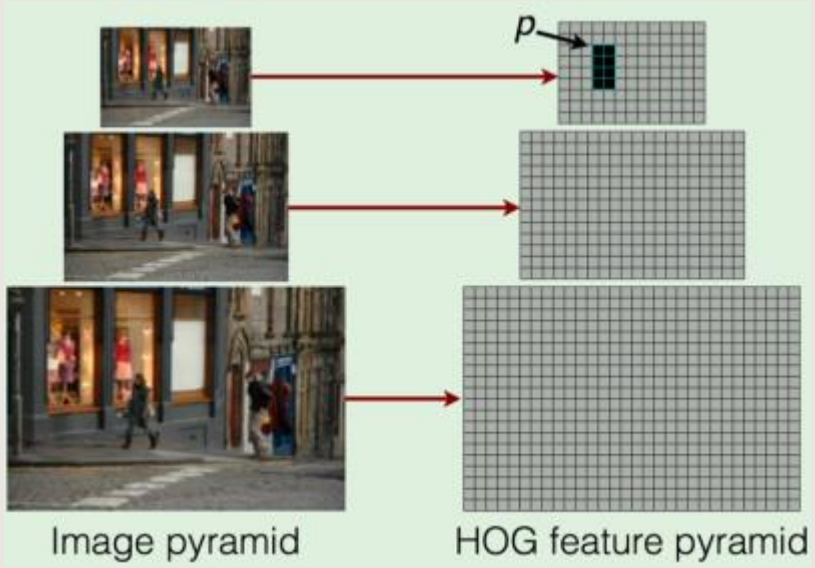
Run linear SVM classifier on all locations

Fuse multiple detections in 3D position & scale space

Object detections with bounding boxes

Train classifier

Predict yes/no of object class in each image window



Pic from: Sanja Fidler

```
# predict
y_pred = model.predict(X_test)
```

Pedestrian detection with HOG - Step by Step

- Scan image(s) at all scales and locations
- Extract features over windows
- Run linear SVM classifier on all locations
- Fuse multiple detections in 3D position & scale space
- Object detections with bounding boxes

Non-maxima suppression to pick the highest box

Pic from: Sanja Fidler

$$overlap = \frac{area(box1 \cup box2)}{area(box1 \cap box2)}$$

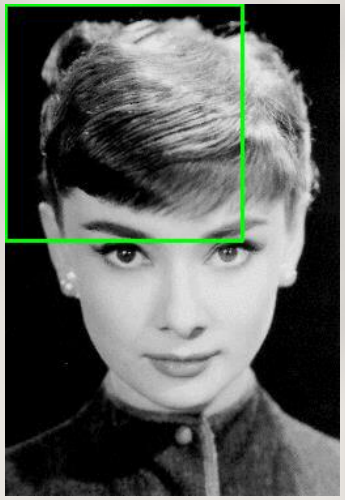
Remove all boxes that overlap more than 0.x (default 0.5) with the chosen box

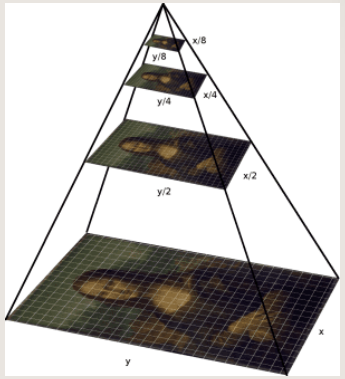
Pedestrian detection with HOG +SVM

```
hog = cv2.HOGDescriptor()
hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())
cv2.startWindowThread()
# open webcam video stream
cap = cv2.VideoCapture(0)
while (True):
    # Capture frame-by-frame
    ret, frame = cap.read()
    # resizing for faster detection
    frame = cv2.resize(frame, (640, 480))
    # using a greyscale picture, also for faster detection
    gray = cv2.cvtColor(frame, cv2.COLOR_RGB2GRAY)
    # detect people in the image
    # returns the bounding boxes for the detected objects
    boxes, weights = hog.detectMultiScale(frame, winStride=(8, 8))
    boxes = np.array([[x, y, x + w, y + h] for (x, y, w, h) in boxes])
    for (xA, yA, xB, yB) in boxes:
        # display the detected boxes in the colour picture
        cv2.rectangle(frame, (xA, yA), (xB, yB),
                      (0, 255, 0), 2)
    # Display the resulting frame
    cv2.imshow('frame', frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
```

Hog.detectMultiScale main parameters

- **winStride**: parameter is a 2-tuple that dictates the "step size" in both the x and y location of the sliding window. The smaller winStride is, the more windows need to be evaluated (which can quickly turn into quite the computational burden).
- **scale**: controls the factor in which our image is resized at each layer of the image pyramid, ultimately influencing the number of levels in the image pyramid. A smaller scale will increase the number of layers in the image pyramid and increase the amount of time it takes to process your image. Typical values for scale are normally in the range [1.01, 1.5]. It allocates image pyramid with 64 levels and scale = 1.05 (default parameters).





Hog.detectMultiScale main parameters

- **scale**: more explained to generate image pyramid

```
# import the necessary packages
import imutils

def pyramid(image, scale=1.5, minSize=(30, 30)):
	# yield the original image
	yield image

	# keep looping over the pyramid
	while True:
		# compute the new dimensions of the image and resize it
		w = int(image.shape[1] / scale)
		image = imutils.resize(image, width=w)

		# if the resized image does not meet the supplied minimum
		# size, then stop constructing the pyramid
		if image.shape[0] < minSize[1] or image.shape[1] < minSize[0]:
			break

		# yield the next image in the pyramid
		yield image
```

```
# METHOD #2: Resizing + Gaussian smoothing.
for (i, resized) in enumerate(pyramid_gaussian(image, downscale=2)):
	# if the image is too small, break from the loop
	if resized.shape[0] < 30 or resized.shape[1] < 30:
		break

	# show the resized image
	cv2.imshow("Layer {}".format(i + 1), resized)
	cv2.waitKey(0)
```

Dataset: https://huggingface.co/datasets/marcelarosalesj/inria-person/tree/main/data_ped.

Summary

- Face detection with Haar-like features using Cascade classifier.
  - Sliding window.
  - Integral image to speed up computation
- Pedestrian detection with HOG features using Linear SVM.